



Project Planning & Prototyping

J. F. N. Salik,

247-519-VA

**Department of Computer
Engineering Technology**
Vanier College
821 Sainte-Croix Avenue
Saint-Laurent, Québec H4L 3X9

This document was last modified on
Thursday 28th May, 2026 at 10:57pm.

<i>Title</i>	Project Planning & Prototyping
<i>Date</i>	May 28, 2026
<i>Author</i>	J. F. N. Salik, (salikj@vaniercollege.qc.ca)
<i>Identification</i>	247-519-VA
<i>Template</i>	Sparkle Version 1.1 for L ^A T _E X 2 _ε



©2026. All rights reserved by J. F. N. Salik,.

This document may not be cited, reproduced, or distributed without explicit
written permission from the author.

Contents

1 Overview: Prototypes, Products & the Business of Engineering	1
1.1 What You Will Build	1
1.2 The Course Map: Two Tracks Over Twelve Weeks	2
1.3 From Prototype to Product: The Gap	3
1.3.0.0.1 In Practice.	4
1.4 Working with Engineers: What You Own	4
1.5 Engineering Businesses by Size	5
1.5.0.0.1 In Practice.	6
1.6 Science for Profit: Cost, Time, and Volume	6
1.7 How a Prototype Becomes Revenue	7
1.7.0.0.1 Common Pitfalls.	9



CHAPTER 1

OVERVIEW: PROTOTYPES, PRODUCTS & THE BUSINESS OF ENGINEERING

Every project begins with a claim: *this idea is worth building*. Your job in Project I is to find out whether that claim is true—cheaply, quickly, and with enough documentation that the answer is defensible. By the end of this course you will have built a working proof-of-concept, assembled the record that shows how you got there, and produced the business case that connects your hands-on work to the real world of engineering firms and markets. This chapter gives you the map. You will see what you are building, why it matters, what separates a prototype from a finished product, and what turns a validated prototype into revenue.

Learning Objectives

1. State what a prototype is for and describe the two deliverables you will produce in Project I.
2. Describe how a proven prototype becomes a product and identify the link between Project I and Project II.
3. Explain your role as a technician working alongside engineers.
4. Explain why industry work is justified by the value it creates, and interpret a negative economic finding correctly.
5. Identify the principal business models that turn a validated prototype into revenue.

1.1 What You Will Build

You will build two things: a **proof-of-concept prototype** and **the record of how you got there**.

The prototype is a working artifact that answers a specific technical and business question. It does not need to be pretty, manufacturable, or certifiable. It needs to

2 OVERVIEW: PROTOTYPES, PRODUCTS & THE BUSINESS OF ENGINEERING

demonstrate that the core idea functions under realistic conditions.

The record is a portfolio of documents—specification, risk register, architecture diagrams, build logs, test data, and cost analysis—that accumulate as you work. You do not write the portfolio at the end. You assemble it from artifacts you produce at each stage. If you work that way, the portfolio is nearly complete before you realize you are making one.

Together, the prototype and its record constitute your output from Project I. The record is as important as the hardware: an engineer can reproduce, evaluate, and extend documented work; undocumented work is disposable.

1.2 The Course Map: Two Tracks Over Twelve Weeks

Figure 1.1 shows the full engineering journey from concept to market. Project I occupies the second stage—the prototype—and Project II the third, where the proven concept is developed into a manufacturable product. The arrow between them is not automatic: what crosses it is the documentation you create in Project I.

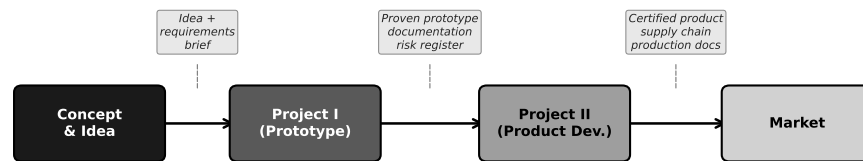


Figure 1.1: The engineering pipeline from concept to market. Project I delivers the proven prototype and its documentation; Project II turns that foundation into a manufacturable product.

The course runs two **parallel tracks**: a theory track and a lab (practice) track. Figure 1.2 shows how they align across the twelve weeks.

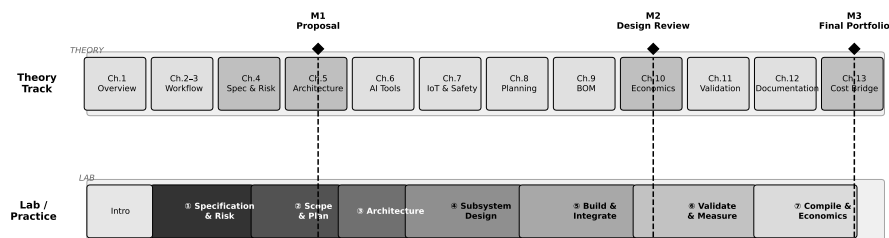


Figure 1.2: The two-track course structure over twelve weeks. The theory track introduces each topic just before the lab requires it; the seven workflow stages unfold across the practice track, with three graded milestones (M1–M3).

The theory track moves through the thirteen chapters of this book at roughly one chapter per week, with Chapters 2 and 3 paired in Week 2. The lab track runs the **seven-stage Prototype Development Cycle**, which you will study in depth

in Chapter 2. At the overview level, the seven stages are:

1. **Specification & Risk** — translate the idea into measurable requirements and a risk ranking.
2. **Scoping & Planning** — decide what you will build, set a schedule, and identify long-lead procurement.
3. **Architecture** — decompose the system into subsystems and define their interfaces.
4. **Subsystem Design** — make detailed design decisions for each subsystem.
5. **Build & Integration** — fabricate and assemble; bring subsystems together.
6. **Validate & Measure** — test against the specification; collect data; decide whether to iterate.
7. **Compile & Economics** — assemble the portfolio and analyse the cost and business case.

Three milestones punctuate the semester. **Milestone 1** (Proposal, end of Week 3) delivers your specification and risk register. **Milestone 2** (Design Review, end of Week 9) delivers your architecture, build records, and preliminary cost analysis. **Milestone 3** (Final Portfolio, end of Week 12) delivers the complete project record.

The theory topics are sequenced so that each week's reading lands just as the lab needs it: you study specifications in Week 3, then immediately write your own in the lab. This is not a coincidence—it is the design of the course.

1.3 From Prototype to Product: The Gap

A prototype and a product solve different problems. A **prototype** answers a question about technical feasibility and business viability. A **product** is an artifact designed to be made reliably, in volume, at a cost that leaves margin, by people who were not involved in the original design, and supported over a service life that may span years.

The gap between them is real and wide. A successful prototype does not automatically become a product. The following properties must be designed in—not bolted on afterward:

Manufacturability. Can the design be made in volume using available processes, tooling, and supply chains? Hand-soldered one-of-a-kind assemblies are not manufacturable at scale.

Reliability. Does the design meet a minimum operating life under realistic field conditions? A prototype that runs for an hour in a lab may fail within a week in the field due to thermal cycling, vibration, or humidity.

Cost at volume. Is the per-unit cost, at the production volumes the business requires, low enough to leave an acceptable margin? Chapter 10 develops the cost model in full.

4 OVERVIEW: PROTOTYPES, PRODUCTS & THE BUSINESS OF ENGINEERING

Certification. Does the design meet applicable electrical safety, radio emissions, and (for IoT and robotics) functional safety standards? Certification is a legal prerequisite in most markets, not an optional finishing step.

Supportability. Can the product be diagnosed and repaired by field technicians? Are replacement parts available over the product lifetime? Is the documentation sufficient for a technician who was not present at the original build?

None of these properties is addressed in Project I. That is not a failure; it is appropriate scope management. Your job in Project I is to answer the central technical and business question as quickly and cheaply as possible. Project II then takes that answer as its starting point and builds the full product around it. What crosses the boundary is your documentation: the specification, the architecture, the risk register, the validated design decisions.

1.3.0.0.1 In Practice. A team building a wearable health monitor completes a functional prototype in twelve weeks: it reads heart rate, displays data on a small screen, and transmits to a phone app. The team is pleased—and surprised, six months later, to learn that the product redesign for manufacturing took an additional eighteen months. The enclosure needed injection-moulding tooling. The radio module required FCC and IC certification. The battery management circuit had to be redesigned for the thermal envelope of a wearable. The prototype proved the concept; the product required a separate, larger effort. Neither effort was wasted.

1.4 Working with Engineers: What You Own

In industry you will work on a team that includes engineers. Understanding who owns what prevents frustration and makes you more effective.

Engineers own:

- Requirements—what the system must do, expressed as measurable criteria.
- Design accountability—the engineer signs off on the architecture and carries legal responsibility for the design.
- Risk decisions—the call to proceed, stop, or redesign rests with the engineer of record.

You own:

- Building—breadboarding, PCB assembly, mechanical fabrication, 3D printing, cable harnesses.
- Testing—executing the test procedure, collecting and recording data honestly, flagging anomalies.
- Documenting—keeping the build log current, maintaining the as-built BOM, photographing assembly steps.

- Procuring—sourcing components, getting quotes, placing purchase orders, receiving and inspecting parts.
- Troubleshooting—diagnosing faults at the bench level.

This boundary is not about status; it reflects training and accountability. A technician who takes over design decisions without the engineer's sign-off creates liability for everyone. A technician who executes, documents, and communicates problems clearly makes the engineer's job tractable. Your hands-on judgment—knowing that a solder joint looks wrong, that a component is getting too hot to touch, that the measured current does not match the schematic—is irreplaceable. Engineers who have lost touch with the bench depend on you for that judgment.

1.5 Engineering Businesses by Size

The same prototype, handed to three firms of different sizes, will be handled very differently. Figure 1.3 illustrates how process formality, documentation weight, and approval layers scale with firm size.

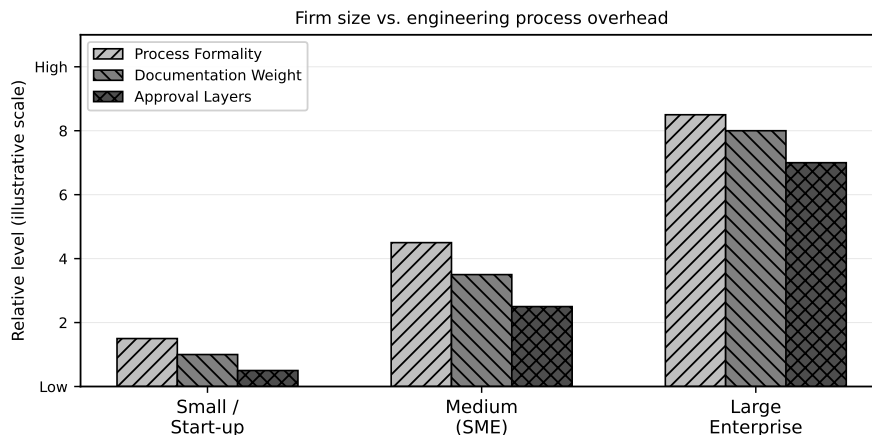


Figure 1.3: Firm-size spectrum: process formality, documentation weight, and approval layers all increase substantially from small start-ups to large enterprises. The prototype development approach must match the firm context.

Small firms and start-ups (under ≈ 20 people) move fast and tolerate ambiguity. A single engineer may own the entire design. Documentation is minimal—often just enough to reproduce the build. The prototype is frequently made by the same people who conceived it. Speed matters more than process. The risk is that institutional knowledge lives in people's heads and disappears when they leave.

Medium firms (SMEs) (20–500 people) have begun to institutionalise. There are defined roles, some formal review gates, and more documentation requirements. A design change needs sign-off from more than one person. The prototype team is likely distinct from the product team, so documentation quality directly affects the handoff.

Large enterprises (500+ people) operate under heavy process—ISO standards, stage-gate reviews, change control boards, formal test plans, and full configuration management. A prototype at a large firm may require months of internal approval

before the first component is ordered. Documentation must be complete, traceable, and auditable.

None of these contexts is inherently better. A start-up that adopts enterprise process collapses under bureaucratic overhead. A large firm that abandons process loses traceability and produces unrepeatable results. Your job is to recognise the context you are in and calibrate your documentation effort accordingly.

1.5.0.0.1 In Practice. A three-person IoT start-up builds a temperature monitoring module in four weeks: breadboard, firmware, cloud logging, mobile app. The “documentation” is a shared folder with photos and a text file. Two years later, the same company has grown to forty people. A new hire tries to reproduce the original circuit for a customer request. The photos are there; the design rationale is not. It takes three weeks to reverse-engineer the component selection. The original four weeks of work left a four-week debt that compounded with every new hire.

1.6 Science for Profit: Cost, Time, and Volume

Engineering work is not science for its own sake—it is science directed at creating value. This does not diminish the intellectual content; it sharpens the questions. In a commercial context, every technical decision carries an economic consequence, and the three most important constraints are **cost**, **time**, and **volume**.

Cost constrains what you can build and at what margin. A solution that is technically elegant but costs twice the target variable cost is not a viable product.

Time constrains competitive advantage. A prototype finished six months late may miss a market window entirely. Time is the one resource you cannot recover.

Volume determines the economic structure. At low volumes, fixed costs (tooling, engineering, certification) dominate the per-unit cost. At high volumes, variable costs dominate. Chapter 10 develops this model formally; here, it is enough to recognise that the prototype’s per-unit cost—often assembled by hand from individual components—tells you nothing about what the product will cost at 1,000 units.

A critical professional skill is the ability to interpret a negative economic finding correctly. Suppose your prototype proves that the design works but the cost analysis shows that no viable margin exists at any realistic production volume. This is not failure—it is the most valuable result the prototype could have produced. You have saved the firm the cost of a full product development effort on an unviable design. Document the finding clearly, state the assumptions, and let the engineering team decide whether to pivot or stop. A prototype that says “no” is doing exactly what a prototype is for.

1.7 How a Prototype Becomes Revenue

Validating a prototype is not the end of the story. The business question that drives the prototype is always, ultimately: how does this create value that someone will pay for? Figure 1.4 maps the major revenue models available once a prototype has been validated.

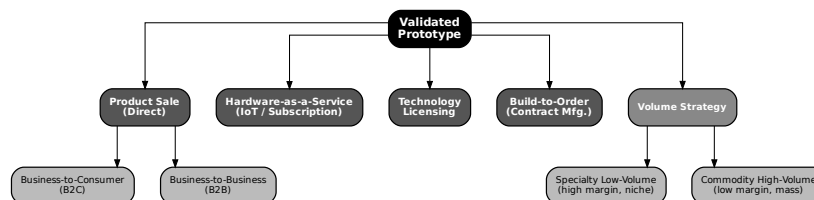


Figure 1.4: Revenue models reachable from a validated prototype. The choice of model shapes what the prototype must prove and what the product development effort must deliver.

Product sale (direct). The most familiar model: manufacture a unit, sell it for more than it costs to make and sell. The two dominant sub-types differ in who the customer is. In a **business-to-consumer (B2C)** model, you sell to individuals; volume is potentially high but margins are typically lower and customer acquisition is expensive. In a **business-to-business (B2B)** model, you sell to companies; volumes are lower but unit prices and margins are generally higher, and relationships carry more weight than brand.

Hardware-as-a-Service (HaaS / IoT subscription). The customer pays an upfront hardware fee and a recurring subscription for connectivity, data processing, and support. The prototype must prove both the hardware function *and* the cloud-connectivity architecture. This model creates recurring revenue and customer lock-in but requires ongoing operational cost for the cloud backend. It is common for IoT devices that produce data the customer values continuously.

Technology licensing. The firm does not manufacture; it licenses the design, firmware, or algorithm to other manufacturers. The prototype proves that the intellectual property is real and valuable. Licensing requires strong IP documentation and often patents.

Build-to-order (contract manufacturing). The firm builds custom units against specific customer purchase orders. Volume is low per customer; the customer drives the specification. Margins can be high because the customer is paying for customisation. The prototype is typically a demonstration to win the contract.

Volume strategy: specialty vs. commodity. Within product sale, the firm chooses where on the volume spectrum to compete. **Specialty low-volume** production targets niche markets with high unit prices and high margins; low volume means less pressure on tooling costs but fewer units over which to amortise fixed costs. **Commodity high-volume** production targets large markets with low unit prices and thin margins; every cent of variable cost matters, and design-for-manufacture is mandatory from the first design decision.

The choice of revenue model is made before the prototype is designed, not

after. It shapes what the prototype must prove, what fidelity is needed, and what documentation must be produced. A HaaS prototype that lacks a working cloud connection has not answered the central business question. A licensing prototype with no IP documentation has nothing to license.

Worked Example: The Sorting Subsystem Business Case

Throughout this book, every chapter works through the same running example: a **connected IoT robotic sorting subsystem**—a sensor-guided actuator that classifies items on a bench conveyor, diverts them to the correct output lane, reports status over a network, and is designed as a module within a larger automation product.

Firm context. The team is a twelve-person engineering start-up that has identified a market in light-industrial sorting for food processing and pharmaceutical packaging. The firm is too small for enterprise process but large enough to have separate engineering and operations functions. Documentation weight is medium: enough to enable reproduction by another team member, not enough to require formal configuration management.

Business model. The chosen model is **Hardware-as-a-Service**: customers purchase the sorter module at a hardware price and pay an annual subscription for cloud-hosted analytics, remote monitoring, and firmware updates. This model requires the prototype to prove two things: (1) the mechanical and electronic sorter works reliably, and (2) the cloud connectivity architecture is sound and can sustain the required data throughput.

What the prototype must answer. The central question is: *Can a sub-\$500 CAD bill-of-materials sorter module achieve $\geq 95\%$ sort accuracy at 6 items/min while maintaining a reliable MQTT connection over a standard Wi-Fi network?* If yes, the hardware is viable and the HaaS model can proceed. If no, the firm needs to know whether the failure is in the mechanical design, the sensing, the firmware, or the network layer—so the portfolio must be structured to show that.

Margin calculation. Suppose the firm targets a hardware unit price of $p = \$1,200$ CAD and estimates the variable cost of a production unit (materials + direct labour at volume) at $c_{\text{var}} = \$480$ CAD. (These are early estimates; Chapter 10 will refine them using the full cost model.)

The unit margin is:

$$m = p - c_{\text{var}} = \$1,200 - \$480 = \$720 \text{ CAD/unit.} \quad (1.1)$$

The gross margin fraction is:

$$\text{GM} = \frac{p - c_{\text{var}}}{p} = \frac{\$720}{\$1,200} = 0.60 \quad (60\%). \quad (1.2)$$

A 60% hardware gross margin is strong for an electromechanical module. It must, however, sustain both the hardware warranty cost and the ongoing cost of the cloud subscription infrastructure. Chapter 10 will introduce the full unit-cost model, $C_{\text{unit}}(N) = C_{\text{fixed}}/N + c_{\text{var}}$, which shows how the fixed non-recurring costs (tooling, firmware development, certification) dilute this margin at low production volumes.

Project context brief (your first portfolio artifact).

- *Project*: IoT robotic sorting subsystem module.
- *Business model*: Hardware-as-a-Service; hardware sale + annual cloud subscription.
- *Firm context*: 12-person engineering start-up; medium documentation weight.
- *Business question the prototype must answer*: Can the module achieve $\geq 95\%$ sort accuracy at 6 items/min with a reliable cloud connection, at a BOM cost below \$500 CAD?

1.7.0.0.1 Common Pitfalls. Technically impressive $\neq$ good prototype. A prototype that demonstrates six different features but answers none of the business questions clearly is not a good prototype. Novelty and complexity are not success criteria. The success criterion is: *did you answer the question?*

"It works" $\neq$ "the business case is proven." A prototype that demonstrates function but carries a BOM cost of \$1,400 CAD against a \$1,200 target price is not a viable product, regardless of how well it works. The economic analysis is not an afterthought—it is half of the prototype's output.

Underrating the technician's judgment. Engineers rely on your observation that the actuator is running hotter than expected, that the network connection drops in the presence of the motor drive, or that a component you received does not match the datasheet. These observations, documented and communicated promptly, are the inputs that prevent costly mistakes. Do not wait to be asked.

Reading a negative economic finding as failure. If the cost analysis shows that the HaaS model is not viable at any realistic volume, that result should be reported clearly, not buried or explained away. The firm can decide to pivot—to a different revenue model, a different component set, or a different market. It cannot decide anything if the analysis is suppressed.

End-of-Chapter Artifact: Project Context Brief

By the end of Week 1, you will produce a one-page **project context brief** for your own project. It is the first document in your portfolio and frames everything that follows. It must contain:

1. A description of the project (one paragraph, no jargon).
2. The chosen business model, with a one-sentence justification.
3. The firm context (size, documentation standard, your role).
4. The single business question the prototype must help answer.
5. A preliminary margin estimate: state p , c_{var} , compute m and GM%, and note the assumptions behind c_{var} .

Keep the brief to one page. Its purpose is not comprehensiveness but clarity: after reading it, a second team member should be able to say whether the project is technically and commercially plausible at a first glance.

Summary

Project I delivers two things: a proof-of-concept prototype and the portfolio that documents how you built and tested it. The prototype's job is to answer one specific technical and business question—not to be a miniature product. Between Project I and Project II lies a deliberate gap: manufacturability, reliability, cost at volume, certification, and supportability must all be designed into the product in Project II, using the prototype's answers as a foundation.

As a technician, you own the build, the test, the documentation, and the procurement. Your hands-on judgment is the most direct input the engineering team has to reality.

Firms handle prototypes differently depending on their size: start-ups move fast with minimal process; enterprises move carefully with heavy documentation. Calibrate your effort to the context.

The economic constraints—cost, time, and volume—are design variables, not external obstacles. A prototype that returns a negative economic result is not a failure; it is evidence. The revenue model chosen before the prototype is built shapes everything: what the prototype must prove, what fidelity is required, and what documentation must be produced.

Up next: Chapter 2 introduces the **seven-stage Prototype Development Cycle** in full. You will see exactly how each stage builds on the previous one, why prototypes answer questions rather than produce things, and how to choose the fidelity level that matches the question at hand—without wasting effort on features the question does not require.



INDEX

B2B, 7
B2C, 7
build-to-order, 7
business model, 7

commodity high-volume, 7
cost, 6

engineer, 4

firm size, 5

hardware-as-a-service, 7

lab track, 2
large enterprise, 5
licensing, 7

milestone, 3

product, 3
product sale, 7
production volume, 6
project context brief, 9
proof-of-concept, 1
prototype, 3

revenue model, 7
running example, 8

seven-stage workflow, 2
SME, 5
specialty low-volume, 7
start-up, 5

theory track, 2
time, 6